pigeon: custody-aware shared folders for small groups

Proposal · 12 October 2026 · supersedes the design memo of the same day

Decision requested. Fund a **two-week feasibility study** (§7) of pigeon as a policy layer over git-annex, with explicit go / no-go criteria. A failure triggers a requirements review, not a custom engine.

Proposal in brief. A small group shares one folder tree peer-to-peer. Laptops keep only the files their owners select, as editable **working copies** that cost about one copy of disk space. Durability comes from **custody copies**: read-only, verified copies held by K **custodians** (machines group members dedicate to storage). pigeon promises precisely that **protected** versions survive, under stated failure and retention assumptions, and it says honestly which versions are not yet protected. On top, it adds **contribution governance**: folders are add-only by default, owned folders change only through their owner, and everyone else's edits become proposals.

1 · Purpose, scope, alternatives

Fits: curated, mostly immutable collections (photos, papers, datasets, archives) larger than any one laptop, plus owner-maintained folders that others contribute to. **Does not fit**: projects edited concurrently by several people (use git or a collaborative editor) or real-time sync of hot files.

Distinctive value, and the only justification for building anything: (1) **custody-aware eviction**: free laptop space without risk, because pigeon knows and verifies where protected copies are; (2) **contribution governance**: add-only by default, owned folders, replicated proposals.

Syncthing + backup	Good sharing; but every device holds full folders (or ignores parts blindly), there is no copy accounting, and no governance.
git-annex assistant	Custody and partial content exist; no governance, no protected/unprotected distinction for thin laptops, git-shaped UX.
Seafile, cloud drives	Server-based; permissions per library; no custody guarantees we control.
Custom engine	Highest NIH risk: re-implements 15 years of git-annex hardening. Only after a requirements review.

2 · Model

Copy classes. A **working copy** lives on a laptop, is editable (thin hard link), and **never counts** toward durability, because an in-place edit can destroy it. A **custody copy** lives on a custodian, is stored read-only, is never exposed to applications, is verified when it is dropped, and **counts**. K = 2 therefore means **two custodians**, not a backup plus a laptop. Custodians are members' machines (a NAS, a desktop, a mini PC); none is central. They may be **partial**: protection is per version, so each holds what its capacity allows, and pigeon only needs K of them to hold any given version.

Version states, shown to users in these words: LOCAL (captured only on this laptop) → PUBLISHED (in the shared history, not yet safe) → PROTECTED (≥ K custody copies on distinct custodians, verified at placement and at the last scrub).

Three different removals, never conflated: **unlink** a path (a new version; protection unchanged) · **evict** a local copy (laptop unsync; allowed only for protected versions) · **release** protection (**purge**: a signed, authorized decision; custodians drop only after the retention period).

Authority. A **roster**, signed from a genesis commit by the founders, lists users and keys, admins, owners per folder, **stewards** of add-only folders, K and the retention period. Changing it requires an admin signature valid under the previous roster. Owned folders accept changes only from their owner. In add-only folders anyone adds, and only stewards decide proposals that modify. A change that breaks the rule becomes a **proposal**; proposals and accept/reject decisions are signed, replicated records. **Force** (any member overriding a rule) and **Drop** (deleting a custody copy without verification) are the only escapes; both are signed, flagged and recorded.

3 · Guarantees

Definitions and assumptions. A version is protected as defined in §2. A **repair epoch** for V ends when V again has K verified custody copies. **F:** within any repair epoch, fewer than K of V's custodians lose or corrupt V. Members are honest but fallible; keys are not stolen. Networks partition, reorder, duplicate and drop; nodes crash at any instant.

- **P1 Survival.** If V is protected and F holds, no sequence of syncs, evictions, laptop edits, crashes or partitions destroys V. Only a purge whose retention period has ended, or Drop, can. **Survival is not reachability**: during a partition V may survive but be out of reach.
- **P2 Capture.** A captured version (committed by pigeon) is never overwritten. Installing incoming content is crash-recoverable and never destroys a file state it has not captured first. Writes made between two captures are protected only on a best-effort basis.
- **P3 Reversible deletion.** Unlinking, including a mass deletion, never releases protection. Until purge, every deleted protected version can be restored. A peer returning from offline cannot resurrect a deleted file or bypass a purge decision.
- **P4 Authority.** The accepted catalogue is the set of commits valid under the roster in force at their parents. Validation is deterministic, so all nodes accept the same catalogue. Commits mixing authorities are split or rejected, and merges are checked against their parents.
- **P5 Convergence.** Identical accepted history gives an identical **logical catalogue** (paths, versions, states). Physical trees differ by selective sync. Names follow one portable rule set: Unicode NFC, case-fold collisions, Windows reserved names, deterministic conflict names.
- **P6 Honest status.** Each version's state is shown as **observed at time T** or **uncertain**. A partitioned node says "unverified since T"; it never claims knowledge it cannot have.

Not guaranteed: LOCAL or PUBLISHED versions not yet PROTECTED (always shown); history beyond retention; malicious members; protection without custodial capacity: a laptops-only group must accept mediated editing or a weaker promise.

4 · Architecture on git-annex

Role	git-annex configuration	Counts toward K
Laptop	unlocked + `annex.thin`; `untrust`; `wanted` from the folders the user selected; `annex.autocommit=false`, `annex.synccontent=false` (pigeon alone commits)	**no**: working copy
Custodian	locked files or bare repository (objects read-only); **semitrusted**, never `trusted` (which skips verification); group `custodian`; `wanted` = `sizebalanced=custodian:K` with a `maxsize` each: partial custodians, balanced by capacity; a custodian keeps what it holds, old versions included (E10)	**yes**
Cloud (later)	encrypted special remote; cannot lock content, so pigeon must serialize drops involving it	yes, with that caveat

Group-wide: `numcopies = mincopies = K`. After every merge pigeon checks that git-annex's shared settings (numcopies, trust, groups, wanted) match the roster, and **suspends all drops** if they do not.

The pigeon daemon (any language; drives `git` and `git-annex`):

1. **Capture.** Watcher → root marker present? → mass-deletion threshold → rule check (an unauthorized edit becomes a proposal and the file is restored) → `git annex add` → signed commit.
2. **Accept.** Fetch peers → verify signature, roster, path rules and merge provenance → merge, or store as a proposal.
3. **Place and repair** (on custodians). Periodic `git annex sync --content --all` and scheduled `fsck`; report repair-epoch progress; alert on capacity shortfalls.
4. **Evict.** Only `git annex drop` (verified; the untrusted laptop never counts), never a plain delete.
5. **Purge.** Signed decision → retention clock → custodians drop the released keys, verified.
6. **Install.** Download to a temp file and verify → journal entry → if the target differs from its last captured state, capture it first → move old aside → rename new in → clear the journal. On Windows, sharing violations mean retry, defer and show, never force.

Transport. ssh on the LAN, iroh across NATs. The published `git-annex-p2p-iroh` helper listens with `dumbpipe listen-unix`, which is compiled on Unix only, so native Windows needs a small iroh helper of our own.

5 · Evidence so far, and its limits

Single-machine runs, git-annex 10.20261006, local remotes (scripts: `lab/git-annex/`). They show the mechanisms behave as documented, **not** that the system is safe.

E1, E5	40/40 concurrent drops of the last two copies left one; a stale location log was not trusted (drop refused).
E2–E6	Deleted content kept until `dropunused`; a propagated `rm -rf` undone by `git revert`; concurrent edits → two `.variant-*` copies; absent content = 102-byte pointer files.
E7	A 60-line pull filter merged an add-only contribution and kept an edit of `alice_/` as a proposal.
E8	`annex.thin`: an in-place edit corrupts the laptop's copy; a later drop elsewhere refused to count it.
E9	**Roles as proposed** (thin untrusted laptop, two custodians, K = 2): a new file stays unprotected and unevictable until both custodians hold it; a custodian's drop is refused if only the laptop would remain; in-place laptop edits leave custody copies intact; a new custodian restored K after one died. **Pitfalls found**: `git annex sync` auto-committed the laptop edit, bypassing capture (→ `autocommit=false`); and the pre-edit version stayed at one copy, because `wanted` = `include=*` covers current files only; `anything` with `--all` restored it to two copies.
E10	**Partial custodians** (3 × 4.5 MB for 10 MB at K = 2): 6/10 files protected, the other 4 shown unprotected and not evictable; after a custodian died, the others repaired within their capacity. Placement is heuristic: 6/10 where 8/10 was possible, even with `--rebalance`.

6 · Risks

Linux: lowest risk, on explicitly supported local filesystems. **macOS**: case and Unicode behaviour, bundles, metadata and app save patterns need a defined scope. **Native Windows**: highest risk. Open-file sharing flags block rename and delete, so retry and recovery are normal paths; a transport helper is needed; WSL results do not count. **UX**: pointer files are not download-on-open placeholders; hiding them needs a second view of the catalogue; users must understand why an unsync waits for custodians. **Capacity**: custodians need an operator, a quota and a replacement plan. Custody needs ≥ K × the collection, and a custodian is useful only up to one copy of everything; git-annex's placement is heuristic (E10), so keep headroom or let pigeon plan placement. **Upstream**: Haskell, one main maintainer, AGPL.

7 · Two-week feasibility study

Week 1 (Linux, macOS). Implement the roles and the six daemon loops above. Fault injection over ≥ 1000 randomized schedules: kill −9 during capture and install, disk full, unplugged drive, partitions, custodian loss and repair. Extend the TLA+ models to the capture/install protocol and to git-annex's lock-based drop. **Week 2 (native Windows + people).** Save-pattern suite (Office, Explorer, VS Code, image editors) under sharing violations; native iroh helper; a real group of 3–4 people with realistic quotas (e.g. 20 GB laptops, 200 GB collection, two custodians), testing whether they understand the three states and proposals.

Go if all hold: zero loss of protected versions in every fault schedule and in the models; Windows capture and install pass the suite; laptop disk ≤ 1.1 × the selected content; test users explain local / published / protected correctly. **Otherwise**: review requirements before building any engine.

Decisions needed now: who hosts the two custodians, and with what capacity · retention period before purge · stewards for add-only folders · roster admins.